

Argumentos *em funções*

Na aula 17, as funções recebiam valores sempre na mesma ordem. Python oferece mais formas de passar argumentos: **pela posição**, **pelo nome**, com **valores padrão** e até argumentos que só aceitam nome. Nesta aula: cada forma, os erros que elas evitam (e causam) e a armadilha clássica do valor padrão mutável.

Posicionais

Nomeados

Valor padrão

TypeError

Padrão mutável

Só por nome (*)

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: More on Defining Functions; FAQ: default values shared between objects), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em [exemplos/](#) · Saídas reais

Pela *ordem*

posicionais.py

```
def apresentar(nome, idade, cidade):  
    print(f"{nome}, {idade} anos, de {cidade}")  
  
apresentar("Ana", 29, "Recife")           # pela ordem  
apresentar("Recife", 29, "Ana")           # ordem errada: sem erro!
```

SAÍDA

```
Ana, 29 anos, de Recife  
Recife, 29 anos, de Ana
```

- ✓ O 1º argumento vai para o 1º parâmetro, o 2º para o 2º...
- ✓ Com a ordem trocada, o Python **não reclama**: o resultado só sai errado
- ✓ Com muitos parâmetros do mesmo tipo, esse risco cresce

"Recife, 29 anos, de Ana": bug silencioso, o pior tipo.
Argumentos nomeados resolvem (próxima seção).

Pelo *nome*

nomeados.py

```
def apresentar(nome, idade, cidade):  
    print(f"{nome}, {idade} anos, de {cidade}")  
  
apresentar(cidade="Recife", nome="Ana", idade=29) # por nome  
apresentar("Bia", cidade="Natal", idade=31)      # misturado
```

SAÍDA

```
Ana, 29 anos, de Recife  
Bia, 31 anos, de Natal
```

- ✓ `parametro=valor` na chamada: a ordem deixa de importar
- ✓ A chamada fica autoexplicativa: `idade=29` diz o que é o 29
- ✓ Pode misturar: **posicionais primeiro**, nomeados depois

Você já usou isso: `print(..., sep="-")` e `sorted(..., reverse=True)` são argumentos nomeados.

Valores *padrão*

padrao.py

```
def saudacao(nome, cumprimento="Olá", pontuacao="!"):
    return f"{cumprimento}, {nome}{pontuacao}"
```

```
print(saudacao("Ana"))
print(saudacao("Bia", "Bom dia"))
print(saudacao("Caio", pontuacao="..."))
```

SAÍDA

```
Olá, Ana!
Bom dia, Bia!
Olá, Caio...
```

- ✓ `parametro=valor` na **definição** torna o argumento opcional
- ✓ Parâmetros com padrão vêm **depois** dos obrigatórios
- ✓ Para pular um opcional e mudar o seguinte, use o nome:
`pontuacao="..."`

Bom para opções que quase sempre têm o mesmo valor:
moeda, idioma, quantidade de casas decimais.

Argumento faltando *ou fora de ordem*

erro_faltando.py

```
def area(base, altura):  
    return base * altura  
  
print(area(3))
```

```
Traceback (most recent call last):  
  File "erro_faltando.py", line 4, in <module>  
    print(area(3))  
          ~~~~^^^  
TypeError: area() missing 1 required positional argument: 'altura'
```

A mensagem diz o nome do parâmetro que faltou: 'altura' .

erro_ordem.py

```
def apresentar(nome, idade):  
    print(nome, idade)  
  
apresentar(nome="Ana", 29)
```

```
File "erro_ordem.py", line 4  
    apresentar(nome="Ana", 29)  
                           ^  
SyntaxError: positional argument follows keyword argument
```

Posicional depois de nomeado não pode: o Python não saberia para onde vai o 29.

Padrão *mutável*

padrao_mutavel.py

```
def adicionar(item, lista=[]):          # armadilha!
    lista.append(item)
    return lista

print(adicionar("a"))
print(adicionar("b"))      # a lista "lembra" da chamada anterior

def adicionar_ok(item, lista=None):
    if lista is None:
        lista = []          # lista nova a cada chamada
    lista.append(item)
    return lista

print(adicionar_ok("a"), adicionar_ok("b"))
```

SAÍDA

```
['a']
['a', 'b']
['a'] ['b']
```

O valor padrão é criado **uma vez só**, quando o `def` roda. Com uma lista, todas as chamadas compartilham a **mesma** lista, que vai acumulando.

- ✓ Nunca use lista, dicionário ou set como padrão
- ✓ Use `None` e crie a coleção dentro da função
- ✓ É a mesma ideia do "=" não copia" da aula 09

Obrigando *a usar o nome*

so_nome.py

```
def transferir(valor, *, de, para):    # depois do * só por nome
    print(f"R$ {valor:.2f}: {de} -> {para}")

transferir(100, de="Ana", para="Bia")
transferir(100, "Ana", "Bia")
```

SAÍDA

```
R$ 100.00: Ana -> Bia
Traceback (most recent call last):
  File "so_nome.py", line 5, in <module>
    transferir(100, "Ana", "Bia")
    ~~~~~^~~~~~
TypeError: transferir() takes 1 positional argument but 3 were given
```

- ✓ Um `*` sozinho na definição: o que vem depois só aceita nome
- ✓ Evita trocar "de" e "para" numa transferência: um erro caro
- ✓ A segunda chamada, sem nomes, é recusada com **TypeError**

O `*` sozinho parece com o `*args` da aula 20, mas aqui ele só marca o limite: não recolhe nada.

Preço final *com opções*

Enunciado: crie `preco_final(preco, desconto=0.0, frete=15.0)` e calcule: sem opções, com 10% de desconto, com frete grátis e com tudo passado pelo nome, em outra ordem.

Uma resposta possível

```
# Exercício: preço final com argumentos padrão
def preco_final(preco, desconto=0.0, frete=15.0):
    return preco * (1 - desconto) + frete

print(f"{preco_final(100):.2f}")
print(f"{preco_final(100, desconto=0.1):.2f}")
print(f"{preco_final(100, frete=0):.2f}")
print(f"{preco_final(frete=0, preco=250, desconto=0.2):.2f}")
```

SAÍDA

```
115.00
105.00
100.00
200.00
```

- ✓ $100 + 15 = 115$; $90 + 15 = 105$; $100 + 0 = 100$
- ✓ $250 \text{ com } 20\% = 200$, frete $0 = 200$
- ✓ Desafio: torne `desconto` e `frete` só por nome

Teste o que *ficou*

1. Em `def f(a, b=2)`, qual chamada é válida?

A) `f()`

B) `f(1)`

C) `f(b=3)`

D) `f(1, 2, 3)`

B. `a` é obrigatório; `b` tem padrão.

2. `f(nome="Ana", 29)` dá:

A) Funciona

B) `SyntaxError`

C) `TypeError`

D) `None`

B. Posicional não vem depois de nomeado.

3. Qual é o padrão seguro para uma lista?

A) `lista=[]`

B) `lista=None`, e criar dentro

C) `lista=list`

D) `lista=()`

B. O padrão é criado uma vez só.

4. Em `def f(x, *, y)`, como passar `y`?

A) `f(1, 2)`

B) `f(1, y=2)`

C) `f(y=2)`

D) Não dá

B. Só pelo nome.

Chamadas claras, *funções flexíveis*

Posicionais seguem a ordem; nomeados deixam a chamada clara e em qualquer ordem; valores padrão tornam argumentos opcionais (e nunca devem ser listas ou dicionários: use `None`). Um `*` sozinho obriga a usar o nome. Erros de chamada aparecem como **`TypeError`** ou **`SyntaxError`**.

Posicional

nome=valor

Padrão

None, não []

def f(x, *, y)

Próxima aula → 19 · Função Recursiva

Fontes: docs.python.org/3/tutorial/controlflow.html#more-on-defining-functions; docs.python.org/3/faq/programming.html (why are default values shared between objects), consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4.